Keywords: pretraining • reinforcement learning • scaling laws • chess reasoning • post-training

Understanding Reasoning from Pretraining to Post-Training

NYU and Collaborators Use Chess as a Controlled Testbed to Uncover a Joint Pretraining-to-RL Scaling Law and Show That RL Concentrates Rather Than Creates Competence

By Jingyan Shen, Ang Li, Salman Rahman, Yifan Sun, Micah Goldblum, Matus Telgarsky, Pavel Izmailov (New York University, Modal Labs, UCLA, UIUC, Columbia University)

arXiv:2607.16097 • July 17, 2026

Reinforcement learning post-training has become a core ingredient in every state-of-the-art language model, yet the field lacks basic answers to two questions: how do pretraining choices shape the returns to RL compute, and what does RL actually do to the model? A team from NYU, Modal Labs, UCLA, UIUC, and Columbia now answers both questions using chess — a fully observable reasoning domain where every move can be evaluated with ground-truth certainty.

AT A GLANCE

Problem: RL post-training is studied in isolation from pretraining, leaving the joint compute allocation problem and RL's mechanistic role undefined.

Why it matters: Understanding the pretraining-to-RL interface is necessary for principled scaling decisions and for knowing what RL can and cannot add to a model.

Method: Pretraining of 5M–1B parameter models on chess games, SFT on synthetic reasoning traces, RL on chess puzzles; move-level probability analysis across checkpoints.

Result: A joint scaling law $R(L_{\text{pre}}, C_{\text{RL}})$ fits held-out runs accurately; RL concentrates probability mass on high-value moves already present in the pretrained distribution.

Evidence: Scaling law extrapolates accurately to a 1B-parameter model with 10× more RL compute than any training run used to fit it.

The Big Picture

Reinforcement learning post-training is now applied to every frontier model, yet the field studies it in isolation: RL is treated as a plug-in module applied to an already-trained base model, with no principled account of how the two training stages interact. This isolation has produced two blind spots. First, practitioners cannot predict how much RL compute a given pretrained model will reward — leading to costly trial-and-error compute allocation. Second, there is no controlled mechanistic account of what RL actually modifies in the model's representations and output distributions; the prevalent intuition is that "RL teaches reasoning" but this is rarely examined at the level of individual predictions.

The paper uses chess as a controlled testbed because it offers ground truth for every decision, a known policy complexity profile, and a dataset of human games spanning all skill levels — ideal conditions for clean scaling experiments that would be impossible on open-ended language tasks.

Why Existing Approaches Fall Short

Prior work on RL for reasoning studies either the RL stage in isolation (holding pretraining fixed) or varies pretraining at large scale without controlling the RL stage. Neither approach can identify the joint interaction between the two stages.

Studying RL in isolation conflates the contribution of the pretrained representation with the contribution of RL optimization, making it impossible to determine whether a performance gain came from RL itself or from the quality of the starting point. Conversely, varying pretraining at fixed RL compute answers which base model RL benefits most, but cannot characterize the functional form of that benefit or reveal what RL does to the model's internal decision-making.

Core Mechanism

The authors vary both pretraining quality (model size and data volume) and RL compute in a factorial design, measuring post-RL reward for each combination. They fit a parametric scaling law to the resulting data and validate it on held-out (pretraining quality, RL compute) pairs. In parallel, they analyze the move-probability distributions of each checkpoint to track how RL reshapes the policy beyond aggregate reward.

Technical Formulation

The joint scaling law expresses post-RL reward R as a function of pretraining loss L_{pre} and RL compute C_{RL}:

$$R(L_{\text{pre}}, C_{\text{RL}}) = R_\infty - \phi(L_{\text{pre}}) \cdot C_{\text{RL}}^{-\gamma}$$

where R_∞ is the asymptotic reward ceiling, $\phi(L_{\text{pre}})$ is a monotonically increasing function of pretraining loss (worse pretraining shifts the ceiling down and slows RL convergence), and γ is a universal power-law exponent

estimated from data.

The function ϕ is fit empirically and found to be approximately linear in L_{pre} over the studied range, giving:

$$\phi(L_{\text{pre}}) \approx \alpha \cdot L_{\text{pre}} + \beta$$

with α and β estimated jointly with γ via nonlinear least squares. This form implies that the marginal value of RL compute is linear in pretraining loss: every unit improvement in pretraining quality proportionally increases the RL compute budget required to reach any given reward threshold.

Learning Procedure

Training proceeds in three stages applied to each factorial combination:

1. **Pretraining:** Next-token prediction on Lichess PGN game records. Models range from 5M to 1B parameters; data volume is varied orthogonally from 10M to 2B tokens.
2. **Supervised fine-tuning:** Training on synthetically generated chain-of-thought traces that articulate candidate move rationale in natural language.
3. **RL post-training:** GRPO with binary correctness reward on chess puzzles; no intermediate-step reward. RL compute is varied from 10^{18} to 10^{20} FLOPs across factorial conditions.

Experimental Findings

The joint scaling law fits held-out (pretraining, RL compute) pairs with relative error below 3% across all studied configurations, confirming that the parametric form captures the essential interaction between the two training stages.

Condition	Predicted R	Observed R
1B params, low RL	0.81	0.80
1B params, high RL	0.89	0.91
70M params, high RL	0.74	0.73
5M params, high RL	0.51	0.52

Mechanistic analysis reveals that RL concentrates the probability distribution: the probability of the ground-truth top move increases by an average of 31% after RL for positions where the pretrained model already ranks it top-1, while the probability assigned to moves not in the pretrained model's top-10 changes by less than 0.3% on average.

Ablations and Interpretation

SFT vs. RL: SFT on reasoning traces reaches a lower performance ceiling than RL, suggesting that imitation of human reasoning traces is insufficient and RL's self-play signal is necessary to reach high puzzle accuracy.

RL without SFT: Applying RL directly to the pretrained model (without SFT) converges more slowly and to a lower final reward, confirming that the chain-of-thought format learned during SFT provides a useful scaffold for RL exploration.

Mechanistic result: RL does not inject knowledge of moves the pretrained model assigned near-zero probability to. This has a direct implication: RL post-training cannot recover from severe pretraining data gaps. A model that has never seen a particular chess strategy in pretraining will not learn it through RL, regardless of RL compute budget.

Scaling extrapolation: The scaling law predicts post-RL reward for a 1B-parameter model trained with 10× more RL compute than any run used to fit the law, with only 1.8% relative error, suggesting the law is reliable for planning compute allocation at larger scales.

Reference

Jingyan Shen, Ang Li, Salman Rahman, Yifan Sun, Micah Goldblum, Matus Telgarsky, Pavel Izmailov. **Understanding Reasoning from Pretraining to Post-Training**. arXiv:2607.16097, July 2026. https://arxiv.org/abs/2607.16097

Keywords: reinforcement learning • compositional reasoning • post-training • skill composition • rewrite grammars

RL Post-Training Builds Compositional Reasoning Strategies

UCL Gatsby Researchers Demonstrate That RL with Only Binary Reward Can Compose Pretrained Primitive Skills Into New Higher-Level Strategies That Rejection Fine-Tuning Cannot Reach

By Azwar Abdulsalam, Nishil Patel, Andrew Saxe (UCL Gatsby Computational Neuroscience Unit)

arXiv:2607.07646 • July 8, 2026

A central question in post-training research is whether reinforcement learning merely amplifies skills already latent in a pretrained base model, or whether it can build genuinely new capabilities by composing those primitives in ways the model has never seen. Researchers at UCL's Gatsby Computational Neuroscience Unit now answer this question decisively using a controlled symbol-rewrite grammar environment: RL with binary final-answer reward not only outperforms rejection fine-tuning on composite tasks, it produces a new compositional mechanism that emerges in two identifiable phases.

AT A GLANCE

Problem: It is unknown whether RL post-training amplifies existing skills or composes them into genuinely new strategies unavailable to the base model.

Why it matters: If RL can only amplify, it is bounded by pretraining; if it can compose, it can transcend pretraining data in principled ways.

Method: Controlled rewrite-grammar environment where primitives and compositions are known; Transformer pretrained on primitives, RL post-trained on composite tasks with binary reward.

Result: RL reaches 78% on composite problems vs. 40% for RFT plateau; compositional strategies emerge in two phases.

Evidence: Phase transition at step ∼600; sequential compositions emerge first, then parallel compositions.

The Big Picture

The dominant post-training recipe for reasoning models involves either rejection fine-tuning (RFT) — filtering model rollouts for correct answers and training on them — or reinforcement learning (RL) with verifiable reward. These two approaches are widely believed to have different capability profiles: RFT is seen as a data-efficient curriculum tool that stays within the distribution of the base model, while RL is believed to discover new strategies through trial and error. But the evidence for this distinction has been largely anecdotal.

The Gatsby team designs a setting where ground truth is available for exactly which strategies the model uses at each step, making it possible to verify mechanistically whether RL produces new compositional behavior or merely amplifies existing primitives.

Why Existing Approaches Fall Short

Both existing post-training approaches have limitations that are difficult to diagnose on open-ended language tasks. RFT cannot improve beyond the base model's rollout distribution: if the base model rarely produces composite solutions, RFT has no signal to amplify. RL can in principle explore beyond the base distribution, but it is unclear whether the gradient signal from binary reward is sufficient to wire together primitive capabilities into structured compositions, or whether it simply boosts the probability of already-composite rollouts that happen to appear at low frequency.

In language settings, these mechanisms are entangled with content and fluency; the rewrite-grammar setting surgically isolates the compositional question from all other confounds.

Core Mechanism

EcoSpec's RL training is shown to operate through a two-phase compositional mechanism. In **Phase 1**, RL sharpens the model's execution of individual primitive rules: the probability of applying the correct primitive to the correct symbol increases, reducing errors in primitive steps. In **Phase 2**, the model begins chaining these primitive applications — first sequentially (applying rule r_i then r_j in a coordinated trace), then in parallel (applying r_i and r_j to independent subterms simultaneously within a single generation). The transition between phases is sharp and reproducible across seeds.

Technical Formulation

The rewrite grammar defines a set of primitive rules $\mathcal{R} = \{r_1, \ldots, r_K\}$. Each primitive r_i rewrites a specific input symbol pattern $s_{\text{in}}^{(i)}$ to $s_{\text{out}}^{(i)}$. A composite problem of arity 2 requires applying rules r_i and r_j either sequentially or in parallel:

$$\text{Sequential:} \quad r_j(r_i(x)) = y \qquad \text{Parallel:} \quad (r_i \,\|\, r_j)(x_1, x_2) = (y_1, y_2)$$

The Transformer is pretrained only on single-rule traces (input, output pairs with r_i applied once). At post-

training, it receives composite problem inputs and binary reward for producing the correct final output.

The policy gradient update under REINFORCE takes the form:

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

where $R(\tau) \in \{0, 1\}$ is the binary correctness reward for the full trace τ and the sum is over all generation steps.

Learning Procedure

1. **Pretraining:** The Transformer is trained on a corpus of single-rule rewrite traces covering all K primitives uniformly. After pretraining, the model reliably applies any single primitive but has near-zero accuracy on composite tasks.
2. **RL post-training:** REINFORCE with binary reward is applied to composite tasks. No intermediate-step reward, no step annotations. Training runs for 2000 gradient steps across all conditions.
3. **RFT baseline:** Rollouts from the pretrained model on composite tasks are filtered for correctness; the model is fine-tuned on the filtered set. RFT training uses the same FLOPs as RL where possible.

Experimental Findings

Method	Accuracy	Plateau
Base model (pass@1)	4.3%	—
Base model (pass@100)	19.1%	—
RFT	40.2%	Step 400
RL (this work)	**78.3%**	Step 2000

The phase transition is visible in trace analysis: the rate of sequential compositions rises from near-zero to ∼30% by step 600, then stabilizes while parallel compositions rise from near-zero to ∼45% by step 2000.

The base model's pass@100 rate (19.1%) establishes a ceiling for RFT: RFT cannot learn from solutions the model never produces under any sampling strategy, yet RFT still substantially surpasses this baseline — suggesting that rare composite rollouts provide sufficient training signal when filtered correctly.

Ablations and Interpretation

Sequential before parallel: Parallel compositions never emerge before sequential ones across any random seed or grammar instantiation. The authors interpret this as a task-induced curriculum: sequential composition is a prerequisite for parallel composition because it requires the model to coordinate rule application across a linear chain before coordinating across independent subterms.

Binary vs. step-level reward: Providing step-level intermediate reward speeds up phase 1 (primitive sharpening) but does not accelerate phase 2 (compositional emergence), suggesting that compositional emergence is driven by global reward signal optimizing the full trace rather than local per-step feedback.

RFT saturation: RFT saturates because the base model's composite-solution rollout rate is low, providing a thin training signal. Once that signal is exhausted, gradient updates from the sparse correct rollouts no longer produce the global policy restructuring that RL achieves through exploration.

Implications: These results support a view of RL as a *composer* rather than merely an *amplifier*: given primitives in pretraining and composition problems at post-training, RL can discover how to chain those primitives into strategies that the base model has never executed.

Reference

Azwar Abdulsalam, Nishil Patel, Andrew Saxe. **RL Post-Training Builds Compositional Reasoning Strategies**. arXiv:2607.07646, July 2026. https://arxiv.org/abs/2607.07646

Keywords: video generation • visual pretraining • diffusion models • perception • general-purpose vision

GenCeption

Google DeepMind Repurposes a Video Generative Diffusion Backbone as a Unified Perception Model, Achieving State-of-the-Art Across Five Vision Tasks with 7–500× Less Training Data

By Letian Wang et al. (Google DeepMind)

arXiv:2607.09024 • July 13, 2026

Computer vision has long operated on a modular paradigm: a separate specialist model for each perception task, each pretrained on its own curated dataset. Google DeepMind's GenCeption challenges this architecture at its root, demonstrating that the spatiotemporal priors baked into a large-scale text-to-video generative model transfer so effectively to perception that a single adapted backbone matches or exceeds task-specific specialists across depth, normals, pose, segmentation, and keypoint prediction — while requiring a small fraction of their training data.

AT A GLANCE

Problem: Vision perception requires separate specialist models per task; no single backbone generalizes across modalities with competitive performance.

Why it matters: A unified perception backbone would reduce maintenance overhead and could exploit shared geometric priors across all tasks simultaneously.

Method: Pre-trained text-to-video diffusion encoder adapted as a feed-forward perception model; task routing via text instructions; lightweight convolutional decoder per task.

Result: State-of-the-art on depth, surface normals, camera pose, segmentation (AP@50 = 61.3), and 3D keypoints (PCK@0.1 = 78.9%).

Evidence: 7–500× data efficiency vs. DepthAnything3, SAM3, VGGT-Omega, Sapiens; emergent synthetic-to-real generalization.

The Big Picture

The dominant paradigm in computer vision treats each perception task as an independent problem: depth estimation, surface normal prediction, camera pose estimation, segmentation, and keypoint prediction each receive a specialist model pretrained on large task-specific datasets. While effective, this paradigm is expensive to maintain and forecloses the possibility of cross-task transfer of geometric knowledge.

GenCeption proposes a radical simplification: the rich spatiotemporal representations learned by a large-scale video generation model — including implicit 3D structure, motion physics, and visual semantics — are precisely the priors that perception tasks require, and can be transferred through minimal adaptation. The key insight is that video generation, to be accurate, must encode depth, surface orientation, object boundaries, and keypoint identity — all the outputs that perception specialists predict — as implicit intermediate representations.

Why Existing Approaches Fall Short

Specialist models excel within their training distribution but fail to leverage shared geometric structure across tasks. A model trained on depth estimation does not automatically learn better surface normal prediction, even though normals and depth are geometrically coupled.

Alternative general-purpose pretraining paradigms — including Video MAE, V-JEPA, and image diffusion models — provide weaker spatial and geometric priors than video generation because they either lack the generative objective (masking-based methods) or operate in 2D rather than the spatiotemporal domain (image diffusion). Video generation requires the model to implicitly solve geometry, physics, and scene understanding at every frame and across temporal transitions, encoding a uniquely rich inductive bias for perception.

Core Mechanism

GenCeption extracts the denoising encoder of a pretrained video diffusion model and adapts it to feed-forward perception through three modifications. First, a **patchification adapter** converts a single image into a sequence of spatial tokens compatible with the video backbone's temporal token representation. Second, **text-steered task routing** injects a task-description embedding via cross-attention at each backbone layer, directing the backbone's feature extraction toward the relevant geometric cues for the requested output. Third, a **lightweight convolutional decoder** maps backbone features to the target output resolution and format (depth map, normal map, segmentation mask, or keypoint heatmap) without adding significant parameter count.

Technical Formulation

Let $x \in \mathbb{R}^{H\times W\times 3}$ be an input image and t a natural-language task description. The video backbone f_θ (frozen then fine-tuned) maps (x, t) to a feature map:

$$F = f_\theta(\text{Patch}(x),\ \text{Enc}(t))$$

where $\text{Patch}(\cdot)$ is the patchification adapter and $\text{Enc}(\cdot)$ is a frozen text encoder. The task decoder g_ϕ then produces

output $\hat{y}$:

$$\hat{y} = g_\phi(F)$$

Training minimizes a task-specific loss $\mathcal{L}(\hat{y}, y)$ (scale-invariant depth loss, angular loss for normals, binary cross-entropy for segmentation, heatmap MSE for keypoints) while the backbone and decoder are jointly fine-tuned. The patchification adapter is initialized from the video model's frame embedding layer and fine-tuned to handle single-image input without temporal context.

Learning Procedure

1. **Backbone source:** A large-scale text-to-video diffusion model pre-trained at Google DeepMind on diverse video corpora is used as the backbone.
2. **Patchification adapter fine-tuning:** Only the adapter is updated for the first 1000 steps to align single-image tokens with the backbone's temporal token distribution.
3. **Joint fine-tuning:** Backbone and decoder are jointly fine-tuned on small per-task labeled datasets (ranging from 1K to 50K examples per task, depending on the task).
4. **Task-unified training:** All tasks are trained jointly with shared backbone weights, with task identity carried entirely by the text prompt.

Experimental Findings

Task (metric)	GenCeption	Specialist SOTA
Depth (AbsRel ↓)	**0.038**	0.041
Normals (mean angle ↓)	**5.2°**	5.8°
Pose (RRA@15 ↑)	**82.4%**	80.1%
Seg. (AP@50 ↑)	**61.3**	59.7
Keypoints (PCK@0.1 ↑)	**78.9%**	76.5%

Specialist comparisons: depth vs. DepthAnything3; normals vs. Lotus-2; pose vs. VGGT-Omega; segmentation vs. SAM3; keypoints vs. Sapiens. Data efficiency is 50× for depth, 100× for normals, 200× for pose, 7× for segmentation, and 500× for keypoints.

Ablations and Interpretation

Backbone type: Replacing the video diffusion backbone with an image diffusion backbone trained on comparable data degrades performance on all tasks by 3–8 points, confirming that spatiotemporal video pretraining — not generative pretraining per se — is the source of the perceptual advantage.

Text routing vs. fixed heads: Ablating text-steered routing (replacing it with fixed task heads) reduces performance by 2–4 points on depth and normals and eliminates zero-shot generalization to task variants described in novel language.

Emergent synthetic-to-real transfer: A model trained exclusively on synthetic human videos achieves competitive performance on real-world footage and on animals — categories entirely absent from training. This behavior is absent when training with image-only or image-diffusion backbones, implicating spatiotemporal video priors as the source of the generalization.

Frozen vs. fine-tuned backbone: Freezing the backbone and adapting only the decoder reduces performance by 8–12 points across all tasks, indicating that the backbone features must be reshuffled to serve the regression objective, not just linearly decoded.

Reference

Letian Wang et al. **Video Generation Models are General-Purpose Vision Learners**. arXiv:2607.09024, July 2026. https://arxiv.org/abs/2607.09024

Keywords: multimodal LLM • open weights • mixture-of-experts • encoder-free • thinking mode

Gemma 4 Technical Report

Google DeepMind Releases Open-Weight Natively Multimodal Models from 2.3B to 31B Parameters, Including an Encoder-Free 12B Architecture Ingesting Raw Audio and Image Patches

By Gemma Team (Sherif El Abd et al.) (Google DeepMind)

arXiv:2607.02770 • July 2026

Google DeepMind's Gemma 4 marks a substantial advance in open-weight multimodal language models, introducing a family spanning 2.3B to 31B parameters across dense and Mixture-of-Experts architectures. The flagship innovation is an encoder-free 12B model that ingests raw image patches and audio frames through the same token embedding space as text — eliminating the modality-specific encoder modules that have been a fixture of multimodal systems since their inception. All models include a built-in thinking mode and are released with full weights, quantized variants, and inference code.

AT A GLANCE

Problem: Open-weight multimodal models lag frontier closed models on STEM, long-context, and audio-visual tasks; separate encoder modules add architectural complexity and maintenance burden.

Why it matters: Open-weight multimodal models are essential for research reproducibility and on-device deployment; removing encoder modules simplifies the architecture fundamentally.

Method: Dense (2.3B, 9B), MoE (26B-A4B), and encoder-free (12B) architectures; thinking mode trained via SFT + RL; 4-bit QAT variants for on-device deployment.

Result: Gemma 4 12B (thinking) reaches 72.1% MMLU-Pro and 91.4% MATH-500; preferred over models 2–3× larger in human-rated evaluations.

Evidence: Evaluated on MMLU-Pro, MATH-500, MMBench, RULER (256K), and speech/audio captioning benchmarks.

The Big Picture

Open-weight language models have narrowed the gap with frontier closed models substantially over the past two years, but multimodal open-weight models have lagged: handling audio natively, processing very long contexts, and generating reasoning traces have each required separate architectural components with separate training pipelines. Gemma 4 unifies these capabilities into a single model family with a consistent architecture and training procedure. The encoder-free 12B model is the most architecturally novel member: by treating image patches and audio frames as tokens in the same embedding space as text, it eliminates the need for a separately pretrained and maintained vision encoder and audio encoder, while retaining competitive performance on both modalities.

Why Existing Approaches Fall Short

Encoder-based multimodal models couple a visual or audio encoder (pretrained independently on large corpora) to a language model through a projection layer. This approach requires maintaining two (or three) separate model components with different training schedules, different fine-tuning protocols, and different quantization sensitivities. Performance is also bounded by the encoder's representational quality: if the encoder is undertrained on certain input types (e.g., audio-visual correspondence), no amount of language model fine-tuning can recover the missing information.

Thinking modes in prior open-weight models have typically been implemented via supervised fine-tuning alone, without RL stabilization, leading to trace inflation — models generating arbitrarily long reasoning traces that do not improve answer quality. Gemma 4 addresses this with a constrained RL stage that optimizes final-answer correctness under a trace-length penalty.

Core Mechanism

The encoder-free architecture tokenizes image patches and audio frames using a learned patchification layer that maps each patch directly to the model's embedding dimension:

$$\mathbf{e}_i = W_p \cdot \mathrm{vec}(p_i) + \mathbf{b}_p$$

where p_i is the i-th patch (image or audio), W_p is the patchification weight matrix, and $\mathbf{b}_p$ is a bias. The resulting token sequence is concatenated with text tokens and processed by a standard causal Transformer, enabling end-to-end attention across all modalities.

Technical Formulation

The MoE model (26B-A4B) uses top-k expert routing with $k = 2$ across $E = 64$ experts. For a given hidden state $\mathbf{h}$, routing scores are computed as:

$$\mathbf{g}(\mathbf{h}) = \mathrm{softmax}(\mathbf{h}W_g) \in \mathbb{R}^E$$

and the top-2 experts are selected and combined:

$$\mathrm{MoE}(\mathbf{h}) = \sum_{i \in \text{top-2}(\mathbf{g})} g_i(\mathbf{h}) \cdot E_i(\mathbf{h})$$

Load balancing is enforced via an auxiliary loss:

$$\mathcal{L}_{\text{aux}} = \alpha \sum_{i=1}^{E} f_i \cdot P_i$$

where f_i is the fraction of tokens routed to expert i and P_i is the mean routing probability to expert i. The thinking mode RL stage applies correctness reward only to final-answer tokens:

$$\mathcal{L}_{\text{RL}} = -\mathbb{E}[R_{\text{correct}} \cdot \log \pi_\theta(a_{\text{answer}} \mid c, \tau)] + \beta \cdot \mathbb{E}[|\tau|]$$

where τ is the reasoning trace, $|\tau|$ is trace length, and β is the trace-length penalty.

Learning Procedure

1. **Pretraining:** All models are pretrained on diverse multimodal corpora including web text, code, image-caption pairs, and audio-speech pairs.
2. **SFT:** Models are fine-tuned on instruction-following datasets and on synthetically generated reasoning traces for thinking mode initialization.
3. **RL:** RLHF with correctness reward for factual and mathematical tasks, combined with the trace-length penalty to prevent trace inflation.
4. **QAT:** Quantization-aware training at 4-bit to produce on-device variants with minimal quality loss.

Experimental Findings

Benchmark	Gemma 4 12B	Previous best
MMLU-Pro	**72.1%**	68.4%
MATH-500 (thinking)	**91.4%**	88.7%
MMBench	**81.6**	79.1
RULER 128K	**97.2%**	94.8%
RULER 256K	87.1%	—
Speech recog. (WER)	**4.3%**	4.5%

The 4-bit QAT variants retain 97–99% of full-precision benchmark scores. Human-rated evaluations prefer Gemma 4 9B and 12B over models with 2–3× more parameters in head-to-head blind comparisons on open-ended generation tasks.

Ablations and Interpretation

Encoder-free vs. encoder-based: On standard image benchmarks, the encoder-free 12B matches the encoder-based 12B within 1–2 points while reducing total parameters and eliminating the encoder maintenance burden.

Thinking mode cost-benefit: Enabling thinking mode doubles average response latency but improves accuracy by 5–15 points on multi-step reasoning tasks. It is implemented as an opt-in feature controlled by the user prompt.

Trace length penalty: Without the trace-length penalty in RL, models generate traces 3–5× longer with no improvement in final-answer accuracy — confirming that unconstrained thinking mode RL leads to trace inflation rather than deeper reasoning.

MoE vs. dense at matched active compute: The 26B-A4B MoE model outperforms a 4B dense model by 4–6 points across all benchmarks, confirming that total parameter count provides independent value beyond active compute.

Reference

Gemma Team (Sherif El Abd et al.). **Gemma 4 Technical Report**. arXiv:2607.02770, July 2026. https://arxiv.org/abs/2607.02770

Keywords: speculative decoding • mixture-of-experts • inference efficiency • expert routing • LLM serving

EcoSpec

Beijing Institute of Technology Identifies Expert Scattering as a Fundamental Barrier to Speculative Decoding in MoE Models and Achieves Up to 1.62× Speedup Through Cost-Aware Draft Selection

By Jincheng Xie, Runheng Liu, Heyan Huang, Yawen Ling, Hanbin Dai, Yu Zheng, Wen Hu (Beijing Institute of Technology)

arXiv:2607.12696 • July 2026

Speculative decoding accelerates autoregressive generation by drafting multiple tokens quickly and verifying them in a single forward pass through the large target model. For dense models this works cleanly: the verification pass has a fixed memory footprint. But for Mixture-of-Experts models — which power all current frontier open-source systems including DeepSeek-V3.1 and Qwen3 — researchers from Beijing Institute of Technology have identified a previously uncharacterized failure mode: *expert scattering*. Their remedy, EcoSpec, jointly optimizes acceptance likelihood and expert reuse, delivering up to 1.62× end-to-end speedup over standard speculative decoding at no quality cost.

AT A GLANCE

Problem: Confidence-driven draft selection in MoE models routes high-probability tokens to disjoint expert sets, increasing expert-weight memory traffic and eliminating the speedup from speculation.

Why it matters: All major frontier open-source models (DeepSeek-V3.1, Qwen3-235B) use MoE; speculative decoding must account for expert memory locality to be effective at scale.

Method: EcoSpec: lightweight expert predictor + dynamic expert buffer; composite draft scoring balancing acceptance likelihood and expert reuse fraction.

Result: Up to 1.62× end-to-end speedup on DeepSeek-V3.1 (671B) across reasoning, coding, QA, and dialogue benchmarks.

Evidence: Standard speculative decoding achieves negative speedup at draft length ≥8 on DeepSeek-V3.1; EcoSpec reaches 1.62× at the same draft length.

The Big Picture

Speculative decoding has become the standard technique for accelerating large language model inference: a small draft model generates a sequence of candidate tokens at low cost, and the large target model verifies the entire sequence in a single forward pass, accepting tokens that agree with its own distribution and rejecting the rest. For dense models, each forward pass has a fixed compute and memory cost, and the speedup is determined solely by the acceptance rate of the draft tokens.

MoE models break this assumption. In an MoE model, each token is routed to a small subset of experts (typically 2 out of 64), and the memory footprint of a forward pass is proportional to the number of *unique* experts activated across all tokens in the batch. When draft tokens are selected to maximize acceptance likelihood alone, those tokens may route to many different expert sets — forcing the serving system to load additional expert weight matrices into GPU high-bandwidth memory for each verification step. This expert scattering can make speculative decoding slower than standard autoregressive generation.

Why Existing Approaches Fall Short

Prior speculative decoding methods for MoE models adopt the same draft selection criterion as for dense models: the draft token with highest likelihood under the draft model is selected at each position. This ignores the routing structure of the target model entirely.

Two prior attempts at MoE-aware speculative decoding — MoESD and Utility-Driven Speculative Decoding — optimize the expert activation pattern for the draft model but not for the target model, leaving the expert scattering problem at the target verification pass unresolved.

Core Mechanism

EcoSpec modifies draft selection by introducing two components. The **expert predictor** is a lightweight auxiliary network that, given the draft model's hidden state for a candidate token, forecasts which of the target model's 64 experts that token will activate. The **dynamic expert buffer** tracks which expert weights are currently cached in GPU high-bandwidth memory for the active verification batch. Draft selection then uses a composite score that balances the probability of token acceptance with the fraction of required experts already in the buffer, so the verification step can proceed with minimal additional expert loading.

Technical Formulation

Let c_i be a candidate token at draft position i and let $p_{\text{accept},i}$ be its acceptance probability under the target model. The expert predictor h_ψ maps the draft hidden

state to a binary vector over the expert set:

$$\hat{e}_i = h_\psi(d_i) \in \{0,1\}^E$$

where d_i is the draft model's hidden state at position i and E is the total number of experts. The expert reuse fraction is:

$$f_{\text{reuse},i} = \frac{|\hat{e}_i \cap \mathcal{B}|}{|\hat{e}_i|}$$

where $\mathcal{B}$ is the current expert buffer (the set of experts in GPU HBM). The composite draft score is:

$$s_i = p_{\text{accept},i} \cdot \left(1 + \lambda \cdot f_{\text{reuse},i}\right)$$

with $\lambda = 0.4$ chosen by grid search. After each verification step, $\mathcal{B}$ is updated to include the experts activated by the accepted tokens.

Learning Procedure

1. **Expert predictor training:** h_ψ is trained offline on a 10K-sample calibration corpus to minimize prediction error for target expert activation patterns. Training takes approximately 30 minutes on a single GPU and adds <0.3% latency to each draft step.
2. **Draft model selection:** Any standard speculative decoding draft model can be used; EcoSpec modifies only the token selection step, not the draft model itself.
3. **Buffer initialization:** At the start of each generation, the buffer $\mathcal{B}$ is initialized with the experts activated by the prompt tokens' verification pass.
4. **No target model modification:** EcoSpec requires no changes to target model weights, architecture, or serving infrastructure beyond the expert predictor and buffer tracking.

Experimental Findings

Model	Task	Std. SD	EcoSpec
DeepSeek-V3.1	Reasoning	1.00×	**1.62**×
DeepSeek-V3.1	Coding	1.00×	**1.51**×
Qwen3-235B-A22B	QA	1.00×	**1.44**×
GPT-OSS-120B	Dialogue	1.00×	**1.38**×

Standard speculative decoding achieves negative speedup (slower than autoregressive) on DeepSeek-V3.1 at draft lengths of 8 or more tokens, confirming expert scattering as a practical barrier. EcoSpec reduces the active expert footprint by 18–35% relative to confidence-only draft selection while maintaining or improving token acceptance rates.

Ablations and Interpretation

Expert predictor quality: Using a perfect oracle predictor improves speedup to 1.81× on DeepSeek-V3.1, indicating that the expert predictor is the current bottleneck and that improving its accuracy is the most promising direction for future gains.

Dynamic vs. static buffer: A static expert buffer (fixed set, not updated per step) achieves only 1.22× speedup — less than half the gain of the dynamic variant — confirming that tracking live HBM state is essential for effective cost-aware draft selection.

Lambda sensitivity: Speedup on DeepSeek-V3.1 varies from 1.41× ($\lambda = 0$, pure confidence) to 1.62× ($\lambda = 0.4$) to 1.48× ($\lambda = 1.0$, pure reuse). The optimal λ is consistent across models and tasks, suggesting that a moderate balance between confidence and reuse is universally beneficial.

Acceptance rate: EcoSpec's composite scoring reduces the acceptance rate by at most 1.2 percentage points relative to confidence-only selection on any tested benchmark, confirming that the speedup comes from reduced expert loading rather than a higher acceptance rate — and that the quality of generated text is unchanged.

Reference

Jincheng Xie, Runheng Liu, Heyan Huang, Yawen Ling, Hanbin Dai, Yu Zheng, Wen Hu. **Less Experts, Faster Decoding: Cost-Aware Speculative Decoding for Mixture-of-Experts**. arXiv:2607.12696, July 2026. https://arxiv.org/abs/2607.12696